

Agente Orientado por Objetivos Usando Deep Q-Learning e PettingZoo para Métodos de Busca

Vitor Leandro Machado, Rodrigo Lucas Rosales

Faculdade de Computação e Informática – Universidade Presbiteriana Mackenzie

São Paulo – SP - Brasil

10409358@mackenzista.com.br, 10365071@mackenzista.com.br

Abstract. *Currently, we have algorithms such as BFS and A* responsible for searching paths in graphs or networks. But what if we consider other environments? Imagine, for example, an autonomous robot responsible for cleaning a space or an autonomous vehicle making deliveries in a city. How would these agents behave? How can we reinforce their learning and measure success rates, efficiency, and effectiveness? Our work proposes creating a navigation agent capable of traversing unknown paths, gradually learning, through reinforcement, to find the optimal route. The result is an intelligent agent that handles unforeseen events, optimizes routes, and moves efficiently from an origin A to a destination B.*

Resumo. *Atualmente, temos algoritmos como BFS e A* responsáveis por buscar caminhos em grafos ou redes. Mas, e se pensarmos em demais ambientes? Imagine, por exemplo, um robô autônomo responsável pela limpeza de um ambiente ou um veículo autônomo realizando entregas em uma cidade. Como esses agentes se comportariam? Como podemos reforçar seu aprendizado e medir taxas de sucesso, eficiência e eficácia? Nosso trabalho propõe criar um agente de navegação capaz de percorrer trajetos desconhecidos, aprendendo gradativamente, por meio de reforço, a encontrar o melhor caminho. O resultado é um agente inteligente que lida com imprevistos, otimiza percursos e se movimenta de forma eficiente de uma origem A até um destino B.*

1. Introdução

Nesta seção separamos a introdução em 5 etapas onde explicamos e contextualizamos algumas capacidades de nosso agente.

1.1 Contextualização

Em um mundo moderno, onde temos cada vez mais ações tomadas por robôs e IA, com o intuito de automatizar tarefas, por que não pensarmos em tarefas intrínsecas ao nosso cotidiano? Como a entrega de um pedido no iFood, o pedido feito no Zé Delivery, ou um robô inteligente que efetua toda a limpeza de uma casa. O tema abordado não é nenhum pouco futurista e já convivemos com ele em alguns ambientes.

A exemplo do robô responsável pela limpeza: compramos uma miniatura de robô inteligente que faz todo o mapeamento do ambiente e atua na remoção de pó e sujeiras; alguns deles até conseguem passar pano no chão. Isso nos mostra o quão poderosos são esses recursos e nos leva a explorar cada vez mais sua capacidade.

O intuito não é substituir uma tarefa humana ou o próprio ser humano, mas sim pensarmos: como posso ter um auxiliar para uma determinada tarefa? E se, por acaso, eu estiver muito ocupado, posso contar com alguém para fazer isto por mim? Esta crítica e pensamento se aplicam à nossa justificativa.

1.2 Justificativa

Podemos explorar recursos que se tornam relevante diante da necessidade de automatizar tarefas que envolvem deslocamento em ambientes complexos ou desconhecidos. Esse tipo de agente aprende a tomar decisões por meio de um processo de acertos e erros, aprimorando continuamente sua capacidade de atingir um destino específico, do ponto A ao ponto B, mesmo diante de obstáculos ou caminhos desconhecidos.

Esse aprendizado adaptativo permite que o agente identifique rotas mais eficientes, evite obstáculos e corrija seus próprios erros, sem depender de intervenção humana constante. Além disso, ele demonstra como recursos inteligentes podem auxiliar em tarefas cotidianas de transporte, entrega ou logística, aumentando a eficiência e reduzindo a possibilidade de falhas.

A criação de um agente de navegação também abre espaço para simulações em ambientes variados, possibilitando estudar comportamentos, testar estratégias de tomada de decisão e explorar soluções que poderiam ser aplicadas em robótica, veículos autônomos ou sistemas de entrega automatizados. Dessa forma, nosso trabalho busca compreender e demonstrar o potencial de agentes inteligentes capazes de aprender e se adaptar

1.3 Objetivo

Criamos um agente que seja capaz de andar por percursos desconhecidos e aprender através de acertos e erros.

A ideia é treinarmos o agente para atingir uma faixa ideal de sucesso, que colocamos como 90% dos caminhos percorridos. Claro que é uma tarefa árdua, dado que o agente teria de conhecer todos os caminhos possíveis para atingir essa faixa de 90%.

Mas pensamos em simular os ambientes sobre um grid, contendo obstáculos. Conforme o agente avança e aprende, pretendemos aumentar o grid, como se estivéssemos ampliando seu mapa de conhecimento, ou seja, o caminho percorrido. A ideia é chegarmos a um grid de 10x10, representando no final uma simulação de entrega nos arredores de uma determinada região.

1.4 Opção do projeto

Optamos por seguir com os percursos em grid, visando o treinamento do agente por meio de grades, aumentando sua complexidade conforme o agente faz o reconhecimento e obtém uma certa taxa de sucesso no ambiente.

2. Fundamentação Teórica

O estudo aborda uma variedade de conceitos e tópicos, que são apresentados de forma resumida nas subseções seguintes.

2.1 Aprendizado por Reforço

O aprendizado por reforço é um processo baseado no mecanismo de tentativa e erro, no qual um agente interage com o ambiente e ajusta seu comportamento a partir das consequências de suas ações. A cada interação, o agente recebe um retorno em forma de recompensa ou punição: recompensas são atribuídas quando a ação realizada contribui para alcançar o objetivo esperado, enquanto punições ocorrem quando a ação conduz a resultados indesejados. Dessa forma, o agente aprende progressivamente a identificar quais estratégias são mais eficazes para atingir sua meta. Com o tempo e a exposição a diferentes cenários, ele aprimora sua tomada de decisão, aumentando a probabilidade de sucesso em situações cada vez mais complexas.

2.2 Agentes Inteligentes

Um agente inteligente é uma entidade que identifica o seu ambiente, toma ações de forma autônoma para atingir objetivos e pode melhorar o seu desempenho através da aprendizagem automática ou adquirindo conhecimento. Segundo **Bringsjord, Selmer; Govindarajulu, Naveen Sunda (2020)** os agentes inteligentes operam com base numa função objetiva, que encapsula os seus objetivos. Eles são projetados para criar e executar planos que maximizem o valor esperado desta função após a conclusão.

Por exemplo, um agente de aprendizado por reforço tem uma função de recompensa, que permite aos programadores moldar e atingir o seu comportamento desejado [Wolchover, Natalie 2020].

O agente inteligente, é aquele que adota a melhor ação possível perante uma situação, estando presente na resolução duma infinidade de problemas dos utilizadores comuns. Hoje, a internet conta com diversas iniciativas que utilizam agentes, desde sites que comparam preços de bens de consumo, a mecanismos de buscas inteligentes, que navegam dentro das páginas web, apresentando o resultado da busca classificado pelo grau de precisão e relevância dos assuntos.

Agentes Inteligentes

Estrutura – Tipos básicos de programas de Agentes – Agentes Reativos Simples

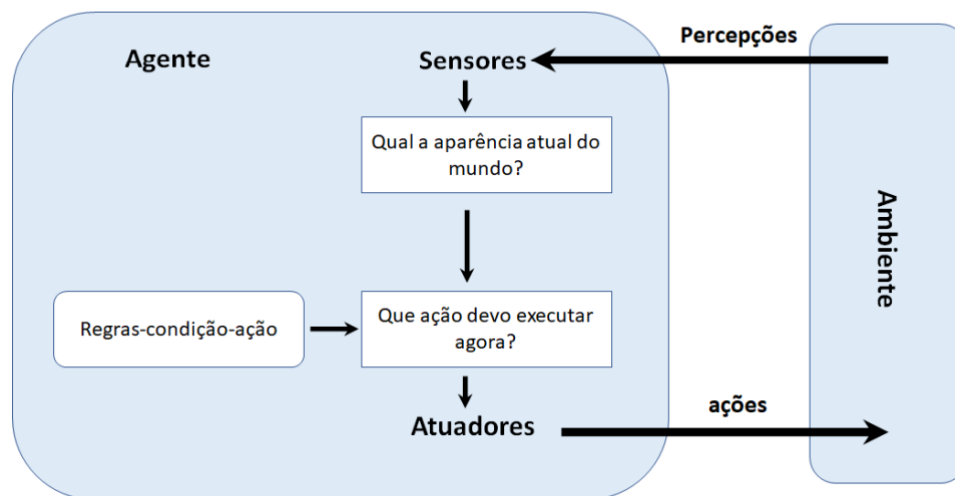


Figura 1 - Imagem estrutura agente inteligente - PDF '02 IA AgentesInteligentes aula.pdf', professor Ivan

2.3 Deep Q-Learning ou Deep Q Network (DQN)

Uma das técnicas mais conhecidas no campo, aprendizado por reforço, é o **Q-Learning**, algoritmo baseado em valores (value-based), no qual o agente aprende uma função de valor de ação $Q(s,a)$, que estima o retorno esperado ao executar a ação 'a' em um estado 's'.

No entanto, o Q-Learning tradicional enfrenta limitações quando aplicado a problemas de alta dimensionalidade, como ambientes com grandes espaços de estados (ex.: jogos de Atari ou ambientes de robótica). Para superar esse desafio, surge o **Deep Q-**

Learning (DQN), é uma extensão do algoritmo básico de Q-Learning. O Deep Q-Learning supera essa limitação substituindo a tabela Q por uma rede neural que pode aproximar os valores de Q para cada par estado-ação

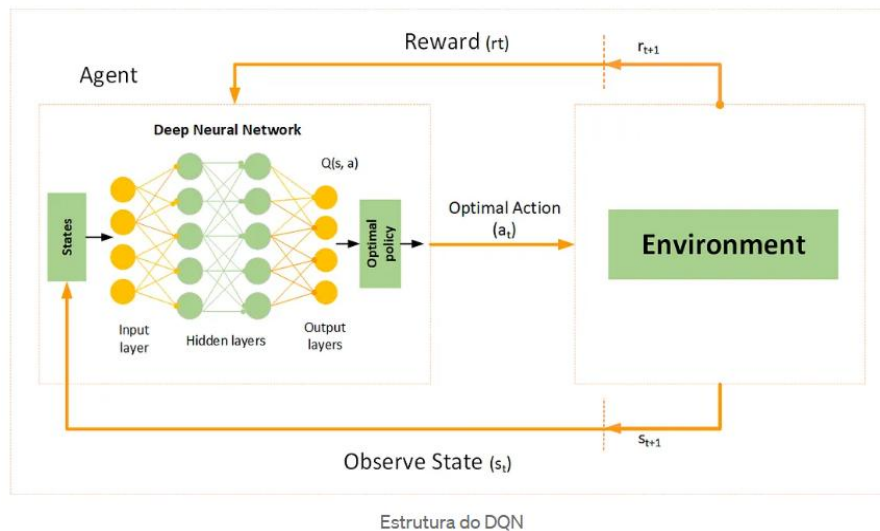


Figura 2 - Rede Deep Q-Learning (DQN) retirada de Amin (2020)

2.4 Treinamento

O agente será treinado através de erro e acerto, seguindo o modelo baseado na aprendizagem por reforço. A ideia é expormos o agente a cenários variados e permitir que ele siga aprendendo conforme erra. Iremos capturar suas métricas e medir sua taxa de sucesso conforme avança.

A ideia é atingirmos um cenário onde o agente consiga atingir o objetivo com maestria e minimizando colisões e optando por caminhos ótimos.

2.5 Petting Zoo

Desenvolvida por Terry et al. (2021), a **biblioteca PettingZoo** é uma extensão da API Gym, criada por Brockman et al. (2016), que adiciona suporte a ambientes multiagente enquanto mantém as funcionalidades e padrões já estabelecidos pelo Gym, incluindo wrappers, gerenciamento de ações, passos e observações. Embora o PettingZoo seja projetado para múltiplos agentes, ele também pode ser utilizado em cenários com **apenas um agente**, mantendo a compatibilidade com a API e permitindo que experimentos sejam facilmente escaláveis para múltiplos agentes no futuro.

O uso do PettingZoo neste trabalho tem como objetivo **padronizar o ambiente**, garantindo compatibilidade com diferentes algoritmos de aprendizado por reforço e mantendo a possibilidade de expansão para múltiplos agentes futuramente. Além disso, a biblioteca facilita a replicabilidade de experimentos, a comparação de resultados e a implementação de wrappers e estratégias de treinamento já existentes na API Gym, mesmo em cenários de agente único.

3. Descrição do problema

Com o avanço das máquinas autônomas e a popularização de agentes de inteligência artificial, surge a necessidade de desenvolver soluções que atuem como aliados do ser humano em diferentes contextos de trabalho e lazer. Um exemplo simples é o robô de limpeza, que auxilia nas tarefas domésticas. Porém, podemos expandir esse conceito para profissões como motoboys, entregadores ou até mesmo investidores: como criar agentes capazes de gerar valor e executar tarefas mesmo enquanto o usuário descansa, viaja ou realiza outras atividades?

Nosso projeto busca explorar esse cenário por meio da implementação de um **agente inteligente baseado em métodos de busca**, capaz de percorrer rotas e caminhos de forma eficiente para resolver problemas específicos de clientes. Um caso prático seria a entrega de produtos, como pizzas ou bebidas (ex.: Zé Delivery), utilizando veículos autônomos (motos ou carros), nos quais o agente atua como responsável pelo planejamento e execução do trajeto de maneira autônoma e otimizada.

4. Ética e Responsabilidades no uso da IA

Nesta seção abordamos como pretendemos aplicar e seguir práticas recomendadas quanto ao uso da IA. Abrangendo como a solução será concebida, treinada, validada e, eventualmente, aplicada em contextos reais.

4.1. Transparência e Explicabilidade

Modelos de IA, especialmente aqueles baseados em aprendizagem por reforço, muitas vezes são percebidos como “caixas-pretas”, tornando difícil compreender os motivos que levam a determinadas decisões.

É fundamental avaliar os riscos potenciais em aplicações críticas — como a navegação em ambientes reais — em que decisões inesperadas podem resultar em comportamentos perigosos.

Nosso objetivo é adotar práticas de desenvolvimento que promovam maior transparência e explicabilidade.

4.2 Segurança e Confiabilidade

Para minimizar falhas e garantir um comportamento seguro do agente, é essencial expô-lo a uma ampla variedade de cenários, incluindo situações extremas, com diferentes níveis de complexidade — baixa, média e alta.

O desempenho do agente será constantemente monitorado para identificar padrões de comportamento inesperados e criar limites de segurança robustos dentro do ambiente de operação. Além disso, serão implementados mecanismos de *fallback* e redundâncias que permitam reagir de forma segura a falhas, protegendo tanto os usuários quanto o entorno.

Essa abordagem proativa não apenas aumenta a confiabilidade do sistema, mas também permite antecipar riscos e fortalecer a resiliência da IA em contextos críticos.

4.3 Privacidade

Em ambientes nos quais a solução coleta dados — como mapas reais, sensores ou câmeras — a proteção da privacidade das informações torna-se uma prioridade.

Para garantir que nenhum dado sensível seja exposto, adotaremos práticas rigorosas de anonimização, além de aplicar o princípio da coleta mínima de dados, capturando apenas as informações estritamente necessárias para o funcionamento do sistema.

Essas medidas asseguram que o uso da IA respeite a privacidade dos usuários, cumprindo normas éticas e regulamentações de proteção de dados.

5. Conteúdo, Origem e Preparação dos Dados

O projeto não conta com um dataset externo tradicional (como imagens ou tabelas). Os dados são **gerados dinamicamente pelo ambiente de simulação** construído em **PettingZoo**, no qual o agente é treinado para navegar em um grid até alcançar um ponto-alvo, desviando de obstáculos.

Cada episódio do treinamento gera trajetórias compostas por **estados, ações, recompensas e próximos estados**. Esses registros equivalem ao que seria um dataset tradicional em problemas supervisionados, mas aqui são produzidos em tempo real à medida que o agente interage com o ambiente.

O conjunto de informações coletadas durante as interações do agente com o ambiente pode ser descrito da seguinte forma:

1. O agente perde ponto a cada passo. Com intuito de que ele siga o caminho mais curto e chegue ao objetivo o mais rápido que puder
2. Caso o agente colida com obstáculos, perderá 5 pontos.
3. Caso atinja o objetivo final, ganhará 100 pontos

A estrutura corresponde, adota ao formato clássico da função Q, do Q-Learning.

$$Q^{\pi}(s_t, a_t) = \underline{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | s_t, a_t]$$

Q-Values for the state given a particular state

Expected discounted cumulative reward

Given the state and action

Figura 3 - Imagem de [freecodecamp.org](https://www.freecodecamp.org)

A análise exploratória ocorre através dos dados gerados por simulação. A medida que o agente é exposto a ambientes e cenários onde precisar encontrar a estratégia ideal para atingir a meta.

Elencamos através de 4 métricas para entender a evolução do agente, conforme é aumentada gradativamente as simulações:

- Distribuição de recompensas ao longo dos episódios (para verificar se o agente está aprendendo);
- Frequência de colisões com obstáculos;
- Quantidade média de passos até atingir o objetivo;
- Taxa de sucesso (% de episódios em que o agente chega ao destino).

6. Resultados

Nesta seção, definimos os critérios de êxito e as metas de desempenho estipuladas para a avaliação do agente de navegação autônoma desenvolvido neste trabalho.

6.1 Capacidade de Navegação e Generalização

O objetivo primário é que o agente demonstre capacidade de generalização, alcançando consistentemente o ponto de destino (B) a partir de uma origem (A) variável. Espera-se que, após o treinamento, a política aprendida convirja para trajetórias ótimas ou subótimas, minimizando a distância percorrida e o custo computacional por episódio.

6.2 Robustez e Evasão de Obstáculo

É fundamental que o agente desenvolva uma política de segurança eficaz, aprendendo a mapear e evitar colisões com obstáculos estáticos. A curva de aprendizado deve refletir uma adaptação progressiva à topologia do ambiente, onde a taxa de colisões apresenta um declínio inversamente proporcional ao número de episódios de treinamento.

6.3 Métricas Quantitativas Esperadas

A validação do modelo será pautada nas seguintes métricas quantitativas:

- Taxa de Sucesso: Atingir um percentual de conclusão de episódios superior a 90% na fase de validação.
- Recompensa Acumulada Média: Obter valores positivos e crescentes, evidenciando a correlação entre as ações do agente e a maximização da função de recompensa.

- **Eficiência de Passos:** Convergência do número médio de passos para valores equivalentes ao caminho mínimo calculado (para este trabalho utilizamos o BFS).
- **Taxa de Colisão:** Redução drástica das penalidades por colisão, tendendo a zero após a estabilização da rede neural.
- Redução da taxa de colisão: idealmente próxima de 0 após convergência.

6.4 Visualização e Evidências

Para validar os resultados, apresentamos uma análise comparativa do comportamento dos agentes em ambientes de complexidade crescente (grids de 5x5, 10x10 e 20x20). O objetivo é demonstrar a disparidade nas curvas de aprendizado entre as abordagens utilizadas.

Enquanto o agente de controle atua de forma puramente estocástica, navegando pelo ambiente sem reter experiência prévia, o agente baseado em Aprendizagem por Reforço (RL) demonstra uma evolução significativa. Os resultados evidenciam como a abordagem de RL permite ao agente transitar de uma exploração inicial para uma política de navegação otimizada, maximizando a eficiência na tomada de decisões e as recompensas acumuladas.

Na figura 4, os logs evidenciam a ineficiência da navegação aleatória. Em um cenário 10x10, o agente necessitou de 2.713 passos para completar uma rota cuja solução ótima seria de apenas 18 passos. Além do alto custo computacional, observa-se um número excessivo de colisões (337), resultando em uma taxa de eficiência de trajeto inferior a 1%. Isso comprova que, sem aprendizado, o agente atua por força bruta, sendo inviável para ambientes complexos.

```

-----
STATUS FINAL      : SUCESSO
Grid Size         : 10x10
-----

Passos Realizados : 2713
Passos Ideais     : 18
Colisões          : 337
Recompensa Total  : 1188.00
Taxa de Sucesso   : 0.66%
=====

```

Figura 4 - Análise do Agente Estocástico

Em contraste, o agente treinado com Deep Q-Network demonstrou convergência para a política ótima. Como observado nos logs de validação, o agente atingiu 100% de eficiência, igualando o número de passos realizados (6 e 8) ao mínimo ideal calculado pelo algoritmo BFS. A ausência de movimentos redundantes e a maximização da recompensa indicam que o modelo foi capaz de generalizar a estrutura do labirinto, planejando rotas diretas e livres de colisões. Onde evidenciamos seus resultados na figura 5 abaixo.

```
=== EPISÓDIO 1 ===
STATUS          : SUCESSO
Grid Size       : 10x10
Passos Realizados : 6
Passos Ideais   : 6
Eficiência      : 100.00%
Recompensa Total : 9.9400
-----

=== EPISÓDIO 2 ===
STATUS          : SUCESSO
Grid Size       : 10x10
Passos Realizados : 8
Passos Ideais   : 8
Eficiência      : 100.00%
Recompensa Total : 9.9200
-----
```

Figura 5 - Análise do Agente com DQN

Após o treinamento, o agente exibe um comportamento proposital. A visualização do grid final e os logs de métricas confirmam que o agente aprendeu a correlação entre sua posição, a posição do alvo e a localização das paredes. O resultado é uma trajetória limpa, onde cada ação aproxima o agente do objetivo, eliminando o desperdício de movimentos observado na abordagem da figura 4 com o agente estocástico.

Para validar a eficácia da política de navegação aprendida pelo DQN, realizamos a inspeção visual da trajetória do agente no grid. A figura abaixo ilustra um episódio de validação onde o caminho mais curto calculado (Meta Ideal) exigia 6 passos.

```
=== EPISÓDIO 1 ===  
Meta Ideal (Caminho Mínimo): 6 passos  
Passos Realizados: 1  
-----  
  . . . . . # . . .  
# . . . . . . . . .  
  . # . . . . . #  
  . . # . . . . # . .  
# . # A . . . . . .  
  . . . . . # . # .  
G . . . . . # . # .  
  . # . . . . . # . #  
  . . . # . . . . .  
# . . . . . . # #  
-----  
Legenda: A = Agente (Verde) | G = Objetivo (Amarelo) | # = Parede | . = Livre
```

Figura 6 – trajetória agente pelo GRID

```
=== FIM DO EPISÓDIO 1 ===  
  . . . . . # . . .  
# . . . . . . . . .  
  . # . . . . . #  
  . . # . . . . # . .  
# . # . . . . . .  
  . . . . . # . # .  
A . . . . . # . # .  
  . # . . . . . # . #  
  . . . # . . . . .  
# . . . . . . # #  
  
SUCESSO! CHEGOU AO OBJETIVO  
Passos totais: 6  
Eficiência do caminho: 100.00%  
Recompensa Total Acumulada: 9.94  
[LOG] Métricas do episódio 1 salvas.
```

Figura 7 – conclusão trajetória agente pelo GRID.

Observa-se que o agente (verde) identifica corretamente o corredor livre de obstáculos até o objetivo (amarelo). A execução confirma a precisão do modelo: o agente completou a tarefa sem movimentos redundantes ou colisões, igualando o desempenho do algoritmo de busca em largura (BFS) usado como baseline, confirmando a robustez do treinamento.

7. Conclusão

Este projeto representa uma evolução significativa em relação à primeira versão do ambiente de navegação em grade. Enquanto o modelo anterior servia como uma etapa exploratória — com um agente agindo de forma totalmente aleatória em um grid —, esta nova implementação amplia tanto a complexidade quanto o realismo do problema.

Abandonamos o comportamento estocástico em favor de um agente treinado com Deep Q-Learning (DQN), capaz de aprender, melhorar sua política e adaptar-se a cenários diferentes a cada episódio. O agente deixa de apenas explorar o ambiente e passa a tomar decisões orientadas a maximizar a recompensa, demonstrando progresso conforme o ϵ diminui e a política se torna mais consolidada.

Entretanto, a implementação evidenciou a complexidade intrínseca ao Aprendizado por Reforço. Alcançar a estabilidade no treinamento não foi trivial, exigindo um esforço considerável na calibração fina de hiperparâmetros. Observou-se uma correlação direta entre o aumento das dimensões do grid e a dificuldade de convergência: cenários maiores expandem exponencialmente o espaço de estados, o que demanda não apenas ajustes criteriosos em taxas de aprendizado e exploração, mas também acarreta um aumento substancial no tempo computacional necessário para que o agente atinja uma política ótima.

As métricas coletadas permitem monitorar esse desempenho de forma precisa, enquanto o armazenamento dos resultados em arquivo possibilita análises posteriores detalhadas. No geral, o projeto demonstra a transição completa de um sistema estático e aleatório para um ambiente dinâmico e um agente inteligente, estabelecendo uma base sólida para estudos futuros em aprendizado por reforço, navegação autônoma e tomada de decisão em ambientes complexos.

8. Endereço Github e Youtube

Nesta seção disponibilizamos os links para o repositório do projeto e o vídeo de apresentação:

Repositório: <https://github.com/Machaddo/mackenzie-IA-agente-de-navegacao>

Apresentação: <https://youtu.be/iEPmAtXYnIw>

8. Referências

- AWAN, Abid Ali. *Uma introdução ao Q-Learning: um tutorial para iniciantes*. DataCamp, 24 abr. 2024. Disponível em: <https://www.datacamp.com/pt/tutorial/introduction-q-learning-beginner-tutorial>. Acesso em: 26 set. 2025.
- LUU, Quang Trung. *Q-Learning vs. Deep Q-Learning vs. Deep Q-Network*. Baeldung, 12 fev. 2025. Disponível em: <https://www.baeldung.com/cs/q-learning-vs-deep-q-learning-vs-deep-q-network>. Acesso em: 26 set. 2025.
- Q-learning. *Wikipedia*. Disponível em: <https://en.wikipedia.org/wiki/Q-learning>. Acesso em: 28 set. 2025.
- KOVALCHUK, Gregory. *A Beginner's Guide to Q-Learning: Understanding with a Simple Gridworld Example*. Medium. Disponível em: <https://medium.com/@goldengrisha/a-beginners-guide-to-q-learning-understanding-with-a-simple-gridworld-example-2b6736e7e2c9>. Acesso em: 28 set. 2025.
- An Introduction to Q-Learning (Reinforcement Learning)*. FreeCodeCamp. Disponível em: <https://www.freecodecamp.org/news/an-introduction-to-q-learning-reinforcement-learning-14ac0b4493cc/>. Acesso em: 27 set. 2025.
- PettingZoo Documentation*. PettingZoo. Disponível em: <https://pettingzoo.farama.org/>. Acesso em: 20 set. 2025.
- AMIN, Samina. *Deep Q-Learning (DQN)*. Medium. Disponível em: <https://medium.com/@samina.amin/deep-q-learning-dqn-71c109586bae>. Acesso em: 27 set. 2025.
- FARAMA. *Gymnasium Basics: Criação de Ambientes*. Disponível em: https://gymnasium.farama.org/introduction/basic_usage/. Acesso em: 28 set. 2025.
- SILVA, Lucas Souza; OLIVEIRA, Ivan Carlos Alcântara de. *Ambiente Multiagente para aplicação de Técnicas de Aprendizado por Reforço na Análise de Fluxo de Atendimento em Locais Fechados*. Trabalho de Conclusão de Curso (Graduação em Ciência da Computação) – Universidade Presbiteriana Mackenzie, São Paulo, 2023. Disponível em: biblioteca mackenzie. Acesso em: 26 set. 2025.
- WIKIPÉDIA. *Agente inteligente*. Disponível em: https://pt.wikipedia.org/wiki/Agente_inteligente. Acesso em: 27 set. 2025.
- STANFORD ENCYCLOPEDIA OF PHILOSOPHY. *Artificial Intelligence (Summer 2020 Edition)*. Disponível em: <https://plato.stanford.edu/archives/sum2020/entries/artificial-intelligence/#InteAgenCont>. Acesso em: 27 set. 2025.

8. Bibliografia

- PYKES, Kurtis. *Reinforcement Learning: An Introduction With Python Examples*. DataCamp. Disponível em: <https://www.datacamp.com/tutorial/reinforcement-learning-python-introduction>. 2025.
- FARAMA. *PettingZoo Documentation*. Disponível em: <https://pettingzoo.farama.org/>. 2025.
- AWAN, Abid Ali. *Uma introdução ao Q-Learning: um tutorial para iniciantes*. DataCamp, 24 abr. 2024. Disponível em: <https://www.datacamp.com/pt/tutorial/introduction-q-learning-beginner-tutorial>. 2025.
- AMIN, Samina. *Deep Q-Learning (DQN)*. Medium, 14 set. 2024. Disponível em: <https://medium.com/@samina.amin/deep-q-learning-dqn-71c109586bae>. 2025.
- NEVES, Enzo Cardeal. *Aprendizado por Reforço #1 — Introdução*. Turing Talks, 23 fev. 2020. Disponível em: <https://medium.com/turing-talks/aprendizado-por-refor%C3%A7o-1-introdu%C3%A7%C3%A3o-7382ebb641ab>. 2025.